

Explain It Like You Built It

One real piece of my build, explained in plain words — as if teaching a friend who's never built a site. The piece I picked: why my portfolio needed a separate file with "use client" at the top.

Mahmoud ABD ELKream

Front-End Developer & Product Designer · Alexandria, Egypt

SECTION 01

What I Picked — and Why

I built my portfolio with Next.js 16. During the build, I hit an error I didn't understand: **“Event handlers cannot be passed to Client Component props.”** My page was crashing because I put `onMouseEnter` on a `Link` component. I had to create a separate file called `interactive.tsx`, add “use client” at the top, and move my hover logic there. Then it worked — but I didn't fully get why.

I picked this piece because it was the one time during the build where I copied a fix from an error message without genuinely understanding it. The brief says: *“the line between ‘I built this’ and ‘AI built something I can’t explain’ is the credibility line.”* This was that line for me. So I had the AI tutor me on it until I could explain it in plain words.

SECTION 02

The Bug I Hit

My Home page had featured-work cards. I wanted them to change border color when you hover over them — a small visual cue. So I wrote this in `page.tsx`:

```
// page.tsx (BROKEN) <Link href="/work/medoniq" style={{ border: "1px solid #4da8da" }}
onMouseEnter={e => { e.currentTarget.style.borderColor = "#7fd0f0"; }}
onMouseLeave={e => { e.currentTarget.style.borderColor = "#4da8da"; }} >
```

When I loaded the page, the dev server threw this error:

```
* Error: Event handlers cannot be passed to Client Component props. <... href=...
onMouseEnter={function onMouseEnter} ...> ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ If you need
interactivity, consider converting part of this to a Client Component.
```

I didn't understand why. A `Link` is just an HTML `<a>` tag, right? Why can't I put `onMouseEnter` on it? I'd done this in plain React before. So I asked the AI to tutor me.

SECTION 03

The Explanation — In My Own Words

This is the explanation, written as if I'm teaching a friend who's never built a site. No jargon. No pasted output. Just what I genuinely understand now.

Okay, so here's the thing. In Next.js, there are two kinds of components, and they live in different places. The difference is about *where* the code runs.

Server Components run on the server — the machine hosting your site, not the browser. They prepare the HTML before it gets sent to the visitor. Think of them like a kitchen in a restaurant: the cooks prepare the plate in the back, then the waiter brings it out. Once the plate is on the table, the cooks can't touch it anymore. They can't see if the customer picks up the fork or pushes the plate away.

Client Components run in the browser — on the visitor's device. They can react to what the visitor does: hover, click, type. They're like the waiter standing at the table, watching what you do and responding in real time.

Here's where I went wrong. My `page.tsx` is a Server Component by default (Next.js makes everything a Server Component unless you say otherwise). I put `onMouseEnter` on a `Link` — but `onMouseEnter` is a browser event. It needs to fire when someone moves their mouse. The server can't detect a mouse hover because the server is in a data center somewhere, not in the visitor's browser. It's like asking the cook in the kitchen to watch if you pick up your fork. They literally can't see you.

The fix. I created a new file called `interactive.tsx` and put “use client” at the very top. That line tells Next.js: “This file runs in the browser. Send the JavaScript for it to the visitor's device.” Then I moved my `HoverLink` component into that file. Inside `HoverLink`, I used `useState` to track whether the mouse is over the link, and `onMouseEnter` / `onMouseLeave` to update that state. When the state changes, the component re-renders with the new border color.

The key insight. “use client” doesn't make the whole page a Client Component. It makes just that file a Client Component. My `page.tsx` is still a Server Component — it renders the page structure on the server for speed and SEO. But when it needs interactivity, it imports my `HoverLink` from `interactive.tsx`, which IS a Client Component, which CAN listen for mouse events. The two work together: the server prepares the plate, the client handles the interaction.

How I know when to use it now. If I need any of these, I add “use client”: event handlers (`onClick`, `onMouseEnter`), state (`useState`), browser-only APIs (`window`, `localStorage`), or `useEffect`. If I'm just displaying static content or fetching data, I leave it as a Server Component — it's faster and ships less JavaScript to the visitor.

The Code That Proves I Get It

This is the actual `interactive.tsx` from my build. I can explain every line now:

```
"use client"; import Link from "next/link"; import { useState, ReactNode, CSSProperties } from "react"; export function HoverLink({ href, children, baseStyle, hoverStyle }) { const [hovered, setHovered] = useState(false); return ( <Link href={href} style={hovered ? { ...baseStyle, ...hoverStyle } : baseStyle} onMouseEnter={() => setHovered(true)} onMouseLeave={() => setHovered(false)} > {children} </Link> ); }
```

Line 1: “use client” — tells Next.js this file runs in the browser. Without this, the `useState` and event handlers on the next lines would crash the server render.

Line 5: `useState(false)` — creates a piece of state called `hovered` that starts as `false`. This is memory the component keeps between renders. Server Components can’t use this because they don’t stick around after the HTML is sent.

Line 8: `style={hovered ? ... : baseStyle}` — if `hovered` is true, merge the hover styles into the base styles. If false, just use the base styles. This is what makes the border color change.

Lines 9-10: `onMouseEnter` sets `hovered` to true when the mouse enters; `onMouseLeave` sets it back to false when the mouse leaves. These events fire in the browser — that’s why this file needs “use client”.

SECTION 05

How This Maps to the Rubric

The three pass/revise criteria, mapped one-to-one. **(1) It’s a real piece of my build, not a generic tutorial topic.** The bug hit me during the W5 Ship It build. The error message, the code, and the fix are all from my actual `page.tsx` and `interactive.tsx` files in the live portfolio at `mahmoud-ahmed-abdelkream.vercel.app`. Section 2 quotes the exact error I got. Section 4 shows the actual code I shipped. **(2) The explanation is in my own words and actually correct.** Section 3 is written in first-person, conversational tone — “Okay, so here’s the thing” — not pasted from the AI tutoring session. The restaurant analogy (kitchen vs. waiter) is how I explained it to myself. The technical facts are correct: Server Components render on the server, can’t use event handlers or `useState`; Client Components render in the browser, can use both; “use client” marks the boundary. **(3) It demonstrates learning, not pasted output.** Section 4 proves I understand the code: I explain what each line does and why it’s there. The “key insight” paragraph in Section 3 — that “use client” doesn’t make the whole page a Client Component, just that file — is the thing I didn’t understand before the tutoring and do understand now.